Name: Fatima ramzan

Roll no: BITF24A029

Semester:4TH

TITLE: SQL Injection Attacks and Their Prevention in Web Applications

1. INTRODUCTION

Web applications are widely used to store and manage user data such as login credentials and personal information. These applications interact with databases using SQL queries. However, if user input is not properly validated, it can lead to serious security issues such as SQL Injection attacks.

SQL Injection is one of the most dangerous web security vulnerabilities that allows attackers to manipulate SQL queries and gain unauthorized access to the database. This project demonstrates how SQL Injection works and how it can be prevented using secure coding practices.

2. PROBLEM STATEMENT

Most web applications are vulnerable to SQL Injection due to improper input validation and insecure SQL queries. Attackers can exploit these weaknesses to bypass authentication systems, access confidential data, and modify or delete database records.

The main problem is:

- Lack of secure coding in database queries
- Direct use of user input in SQL statements
- No input validation or filtering

This project aims to demonstrate these vulnerabilities and provide effective solutions.

3. OBJECTIVES

- To understand SQL Injection attacks in web applications
- To develop a simple login system using HTML, PHP, and MySQL

- To demonstrate how SQL Injection works in vulnerable systems
- To implement prevention techniques using secure coding
- To improve awareness of web application security

4. SYSTEM DESIGN

The system consists of three main components:

1. Frontend (HTML)

- Login form for user input (username & password)

2. Backend (PHP)

- Processes user input
- Executes SQL queries
- Handles authentication logic

3. Database (MySQL)

- Stores user credentials
- Validates login information

🔄 System Flow:

User Input → HTML Form → PHP Script → SQL Query → Database → Result Display

5. IMPLEMENTATION DETAILS

The project is implemented using:

- **HTML** for creating login interface
- **PHP** for backend logic and database connection
- **MySQL** for storing user data
- **XAMPP** as local server environment

Two versions are implemented:

❌ Vulnerable Version:

- Direct insertion of user input into SQL queries
- Allows SQL Injection attacks

Secure Version:

- Uses prepared statements
- Prevents SQL Injection by separating data from code

6. FEATURES

- Simple login system
- Database connectivity using MySQL
- Demonstration of SQL Injection attack
- Secure authentication system
- Prevention using prepared statements
- Local server execution using XAMPP

7. RESULTS & TESTING

✓ Vulnerable System:

- SQL Injection successfully bypasses login
- Shows how attackers manipulate queries

✓ Secure System:

- SQL Injection attacks are blocked
- Only valid credentials are accepted
- System becomes secure and stable

This comparison clearly shows the importance of secure coding practices.

8. CONCLUSION

SQL Injection is a critical security threat in web applications. It occurs due to improper handling of user input in SQL queries. This project demonstrates both vulnerable and secure systems, highlighting how attacks occur and how they can be prevented.

By using prepared statements, parameterized queries, and input validation, SQL Injection attacks can be effectively prevented. Secure coding is essential for protecting web applications and user data.